

DNS & Subdomain Enumeration

DNS enumeration is the natural next step after WHOIS — where WHOIS tells you *who owns* a domain, DNS enumeration tells you *what actually exists* on it. Subdomains are one of the richest attack surfaces in modern pentesting, since organizations routinely forget about dev/staging/legacy subdomains that never got the same security hardening as the main site.

1. DNS Fundamentals Refresher

DNS translates human-readable names (`example.com`) into IP addresses, and stores several other record types that reveal useful recon information:

Record Type	What It Contains	Recon Value
A	Domain → IPv4 address	Direct target IP
AAAA	Domain → IPv6 address	IPv6 attack surface (often less monitored)
CNAME	Alias → another domain	Reveals third-party services (e.g., <code>shop.example.com → shopify.com</code>)
MX	Mail server records	Reveals email provider (Google Workspace, Office365) — useful for phishing infra planning in authorized engagements
NS	Authoritative name servers	Reveals DNS provider (Cloudflare, Route53)
TXT	Arbitrary text — SPF, DKIM, domain verification	Reveals third-party service integrations (e.g., <code>google-site-verification</code> , <code>_dmarc</code>)
SOA	Start of Authority — zone admin info	Zone serial number, refresh/retry timers
PTR	IP → domain (reverse DNS)	Maps IPs back to hostnames
SRV	Service location records	Reveals internal services (e.g., AD's <code>_ldap._tcp</code>)

2. Basic DNS Lookup Tools

nslookup (cross-platform, basic)

```
nslookup example.com
nslookup -type=MX example.com
nslookup -type=TXT example.com
nslookup -type=NS example.com
```

dig (Linux/Kali — more powerful, preferred for real work)

```
dig example.com                # default A record lookup
dig example.com MX             # mail servers
dig example.com TXT            # text records
dig example.com NS             # name servers
dig example.com ANY            # all record types (often b
locked/rate-limited now)
dig +short example.com         # clean output, just the an
swer
```

Reverse DNS lookup (IP → hostname):

```
dig -x 8.8.8.8
nslookup 8.8.8.8
```

Querying a specific DNS server directly (not your default resolver):

```
dig @8.8.8.8 example.com
dig @ns1.example.com example.com # ask the authoritative
server directly
```

3. Zone Transfer (AXFR) — The Classic Misconfiguration

A **zone transfer** is meant to let a *secondary* DNS server sync its records from the *primary* authoritative server. If misconfigured to allow transfers from **any** IP (not just trusted secondaries), anyone can dump the entire DNS zone — every subdomain, internal hostname, and IP in one request.

```
dig axfr @ns1.example.com example.com
```

If vulnerable, this returns the **complete** internal DNS zone in one shot — historically one of the highest-value, lowest-effort recon wins possible, though modern DNS providers rarely leave this open.

```
# Check all name servers for zone transfer vulnerability
for ns in $(dig +short NS example.com); do
    echo "Testing $ns"
    dig axfr @$ns example.com
done
```

Why this is rare today: most orgs now use managed DNS (Cloudflare, Route53) with AXFR disabled by default. Still worth checking on every engagement — legacy or self-hosted DNS servers sometimes still have it open.

4. Subdomain Enumeration — Why It Matters

Subdomains matter because:

- **Dev/staging environments** (`dev.` , `staging.` , `test.`) are often less patched, sometimes have debug mode enabled, or use default credentials
- **Forgotten legacy systems** (`old.` , `legacy.` , `vpn-old.`) may run outdated, vulnerable software
- **Shadow IT** — subdomains spun up by individual teams without security review (`marketing-app.` , `intern-project.`)
- Each subdomain is potentially a **separate attack surface** with its own ports, services, and vulnerabilities

5. Passive Subdomain Enumeration (No Direct Contact With Target)

Passive methods query third-party data sources instead of touching the target's infrastructure directly — lower risk, often done before any active engagement begins.

a) Certificate Transparency Logs (extremely high-value technique)

Every publicly-trusted SSL/TLS certificate is logged in public **CT logs** — since certificates include the domain name (and often subdomains via SAN fields), searching CT logs reveals subdomains even if they're never linked anywhere on the actual website.

```
# crt.sh - free web-based CT log search
# Visit: https://crt.sh/?q=example.com
```

```
# Or query it via curl and parse with jq
curl -s "https://crt.sh/?q=example.com&output=json" | jq
-r '[][.name_value]' | sort -u
```

This is one of the single most effective passive subdomain discovery techniques — it works because certificate issuance is *mandatory* for HTTPS, so even a subdomain never linked publicly still leaves a trace the moment a cert is issued for it.

b) Sublist3r

```
sublist3r -d example.com
sublist3r -d example.com -o subdomains.txt
```

Aggregates results from multiple sources — search engines (Google, Bing, Yahoo), Netcraft, VirusTotal, DNSDumpster, and crt.sh — all in one tool.

c) Amass (OWASP project — the industry-standard tool)

```
amass enum -d example.com
amass enum -passive -d example.com # passive-only mode,
no active probing
amass enum -d example.com -o results.txt
```

Amass is considerably more thorough than Sublist3r — pulls from dozens of data sources (CT logs, WHOIS, DNS, scraping, APIs) and can also do active enumeration (brute-forcing, DNS resolution validation).

d) Assetfinder

```
assetfinder --subs-only example.com
```

Lightweight, fast, good for quick passive checks — often chained with other tools in a pipeline.

e) VirusTotal / SecurityTrails / DNSdumpster (web-based)

These platforms maintain historical DNS resolution data — useful for finding subdomains that existed in the past even if they're no longer active, and for spotting infrastructure patterns.

6. Active Subdomain Enumeration (DNS Brute-Forcing)

Active methods directly query DNS servers with a wordlist of common subdomain names — noisier, but finds subdomains that passive sources never picked up (because they were never certificate-issued or search-indexed).

a) **gobuster** (DNS mode)

```
gobuster dns -d example.com -w /usr/share/wordlists/subdomains.txt
gobuster dns -d example.com -w subdomains.txt -t 50 # 50 threads
```

b) **dnsrecon**

```
dnsrecon -d example.com -t brt -D /usr/share/wordlists/subdomains.txt
dnsrecon -d example.com -t std # standard enumeration (SOA, NS, MX, zone transfer check, etc.)
```

c) **dnsenum**

```
dnsenum example.com
dnsenum --dnsserver 8.8.8.8 -f subdomains.txt example.com
```

d) **ffuf** (fast, flexible, uses HTTP Host header fuzzing — catches subdomains behind virtual hosting)

```
ffuf -w subdomains.txt -u https://FUZZ.example.com -mc 200,301,302
```

Why combine passive + active: passive finds subdomains that exist in public data sources; active brute-forcing finds subdomains that are DNS-resolvable but were never publicly recorded anywhere — the two approaches cover different blind spots.

7. Validating Discovered Subdomains (Filtering Noise)

Enumeration tools often return subdomains that no longer resolve or are dead — a critical next step is checking which ones are actually **live**:

```
# httpx - checks which subdomains respond over HTTP/HTTPS
cat subdomains.txt | httpx -silent -status-code -title

# httprobe - similar purpose
cat subdomains.txt | httprobe
```

This turns a raw list of hundreds of subdomains into a filtered list of "these are actually alive and worth investigating further" — an essential step before moving to port scanning/service enumeration, since scanning dead hosts wastes time.

8. Wildcard DNS — A Common False-Positive Trap

Some domains configure **wildcard DNS** (`*.example.com` resolves to the same IP for literally any subdomain), which causes brute-force tools to report thousands of "found" subdomains that don't actually exist as distinct services.

Detecting a wildcard:

```
dig random-nonexistent-subdomain-12345.example.com
```

If this resolves to an IP anyway, the domain has wildcard DNS enabled — most modern enumeration tools (Amass, dnsrecon) automatically detect and filter this out, but it's important to understand *why* your results might be full of noise if you're seeing suspiciously high subdomain counts.

9. Putting It Together — A Practical Recon Workflow

```
# 1. Passive - certificate transparency
curl -s "https://crt.sh/?q=example.com&output=json" | jq
-r '[][.name_value]' | sort -u > ct_subs.txt

# 2. Passive - Amass
amass enum -passive -d example.com -o amass_subs.txt

# 3. Active - brute force with gobuster
gobuster dns -d example.com -w subdomains.txt -o brute_sub
s.txt

# 4. Combine and deduplicate
cat ct_subs.txt amass_subs.txt brute_subs.txt | sort -u > a
ll_subs.txt

# 5. Validate which are alive
cat all_subs.txt | httpx -silent -status-code -title -o liv
e_subs.txt

# 6. Resolve to IPs for port scanning
cat live_subs.txt | dnsx -a -resp-only > resolved_ips.txt

# 7. Feed into Nmap
nmap -iL resolved_ips.txt -sV -oN subdomain_scan.txt
```

This exact chain — CT logs → Amass → brute force → httpx validation → Nmap — is close to the real-world methodology used in professional bug bounty and pentest recon phases, and would map cleanly onto a "Reconnaissance" section of a VAPT report.

10. DNS/Subdomain Enum in the Kill Chain / ATT&CK Context

Framework	Mapping
Cyber Kill Chain	Reconnaissance stage
MITRE ATT&CK	T1590.002 (Gather Victim Network Information: DNS), T1596.001 (Search Open Technical Databases: DNS/Passive DNS)

Worth tagging your own recon notes with these IDs as you practice — reinforces the habit from the ATT&CK guide earlier.

11. Defensive Angle (SOC Relevance)

- **DNS query logging** — SOC teams monitor for unusual DNS query patterns (e.g., a single internal host suddenly resolving hundreds of random-looking domains = possible DGA malware/C2)
- **Subdomain takeover risk** — if a subdomain's CNAME points to a third-party service (like an AWS S3 bucket or Heroku app) that's since been deleted, an attacker can claim that service and effectively hijack the subdomain

This is a very commonly reported bug bounty finding category — worth knowing as a distinct concept from basic subdomain enumeration

```
# Check for dangling CNAMEs pointing to deprovisioned services
subjack -w subdomains.txt -t 50 -timeout 30 -o results.txt -ssl
```

- **Attack surface management (ASM)** — many organizations now run continuous automated subdomain discovery against themselves (using the same Amass/crt.sh techniques) specifically to catch shadow IT before attackers do

Quick Reference Cheat Sheet

```
dig example.com                # A record
dig example.com MX/TXT/NS     # other record types
dig -x 8.8.8.8                # reverse lookup
```



```
dig axfr @ns1.example.com example.com # zone transfer attempt

curl -s "https://crt.sh/?q=example.com&output=json" | jq
-r '[][.name_value]' | sort -u # CT logs

sublist3r -d example.com # multi-source passive
amass enum -passive -d example.com # thorough passive
gobuster dns -d example.com -w list.txt # active brute force
cat subs.txt | httpx -silent -status-code # validate live
subjack -w subs.txt -ssl # check subdomain takeover risk
```